— *Solution notes* —

COMPUTER SCIENCE TRIPOS Part IB – 2021 – Paper 4

2 Programming in C and C++ (djg11)

C reference machine

(*a*) A key/value store holds pairs of strings in a non-volatile memory (NVM). The NVM is mapped as a memory region between two hardcoded virtual addresses, NV_START and NV_END. This memory is all set to zero as manufactured and can be freely read, but any write operations result in a logical ORing of the new data with the old data at the addressed location. Bits can never be cleared. Strings representing keys and values will be fewer than 250 characters long. Eventually the memory will fill up, but sufficient is available for the intended application.

The API provides the following two operations:

```
int store(char *key, char *value) // returns non-zero on error, and
char *lookup(char *key)           // returns NULL if not found.
```

Provide a full C implementation of the `store` routine, explaining how the lookup function and changes to values under a given key would work, if this is not obvious from your code. Code efficiency is not important.
Hint: You can directly access the non-volatile store using a construct such as `((char *)NV_START)[offset]`. [10 marks]

Answer: I will use linear store allocation and a three byte header for each record where the first byte is zero for virgin space, 1 for in use and 3 for deleted. The second and third fields are the string lengths. We don't need to store the trailing null of each string, but I do.

```
unsigned char r8(int o) {return ((unsigned char *)NV_START)[o];}
void w8(int o, unsigned int v) { ((unsigned char *)NV_START)[o]= v;}

int store(char *key, char *value)
{ int p = 0, kl = strlen(key), vl = strlen(value);
  if (kl>=250 || vl>=250) return -2;
  while (1)
    { if (r8(p)==0) break; // Found virgin ground.
      if (r8(p)==1&&r8(p+1)==kl && !strcmp(key,(char*)(NV_START+p+3)))
        { // printf("Delete old entry at %i\n", p);
          w8(p, 3);
        }
      p += 3 + r8(p+1) + r8(p+2) + 2;
    }
  if (p > NV_END-NV_START - 3 - kl - vl) return -1;
  w8(p, 1); w8(p+1, kl); w8(p+2, vl);
  for (int i=0; i<=kl; i++) w8(p+3+i, key[i]);
  for (int i=0; i<=vl; i++) w8(p+3+kl+1+i, value[i]);
  return 0;
}
```

References, constructors, assignment overloading

(*b*) The following text *almost* constitutes both a C++ and Java program:

```
class Foo { public: int x, y; };
```

```
class Test {
  private: void f1(Foo p) { p.x = 1; }
  private: void f2(Foo &q) { q.y = 2; }  //[Java]: ignore '&'
  public: void test() {
    Foo p;  //[Java]: replace with: 'Foo p = new Foo();'
    p.x = p.y = 99;
    f1(p);
    Foo q = p;
    p = q;
    f2(q);
    print("HERE");
  }
};
```

(*i*) Explain the storage structure accessible from variables `x` and `y` when control reaches `print("HERE")` following a call to `test()`, both in the Java and the C++ interpretations. [3 marks]

Answer: The answer centres on the fact that `Foo p;` and `Foo q;` declare references in Java but structure values in C++. At `print("HERE")` in Java both `p` and `q` are references to the same heap-allocated structure which has fields `x` containing 1 and `y` containing 2. In C++ both `p` and `q` are stack-allocated structs containing two integer fields. We have $\texttt{p.x} = 99, \texttt{p.y} = 99, \texttt{q.x} = 99, \texttt{q.y} = 2$. Because the parameter to `f1` is call-by-value the assignment `p.x = 1` has no effect on the caller function `test`.

(*ii*) For the C++ interpretation, show how adjustments *only to the definition* of `class Foo` can cause (useful) debugging-style output to be produced at *as many places as possible* during the execution of method `test()`. [7 marks]

Answer: [*Note:* some students noted that changing the type of `x` and `y` from `int` to `MyInt` would enable writes such as `p.x = 99` also to be tracked. This did not get additional marks, and perhaps the question should have said "*only to the methods* of `class Foo`".
Similarly, "*as many places as possible*" should ideally have been "*as often during execution as possible*".]

```
class Foo { public: int x, y;
  Foo() { printf("construct %p\n", this); }
  Foo(Foo &r) {
    printf("copy construct %p(%d,%d)\n", this, r.x, r.y);
    x = r.x, y = r.y; }
  ~Foo() { printf("destruct %p(%d,%d)\n", this, x, y); }
  void operator= (Foo &r) {
     printf("assign fields %p = %p(%d,%d)\n", this, &r, r.x, r.y);
     x = r.x, y = r.y;
  }
};
```

Because the question asked for debugging-style output, this code makes no change to the behaviour of the program. [*Note:* There are many 'interesting' adjustments of the

code which cause the values at line `print("HERE")` to be very different!]

Numbering the lines with line 1 corresponding to `Foo p;` we have: One mark for spotting the call to default constructor on line 1, one mark for two destructor calls at line 8 (scope end), two marks each for overloading assignment at line 5, and for copy constructor at line 4. A final mark for noting the copy constructor (and destructor) needed at the call to `f1` on line 3. A smart student might note that line 6 `f2(q)` is not trackable by merely adjusting `Foo`, but no additional mark for this; such students instead preferred to note that the `MyInt` approach above could track the assignment within `f2`. [*Note:* C++ does not allow the '`.`' field-selection operator to be overloaded.]

For clarity below (no marks for this), I have replaced line 7 `print("HERE");` with

```
printf("HERE %d %d %d %d\n", p.x, p.y, q.x, q.y);
```

Example output:

```
construct 0x7ffee26f2ac8 [line 1]
copy construct 0x7ffee26f2ac0(99,99) [line 3]
destruct 0x7ffee26f2ac0(1,99) [line 3/body of f1]
copy construct 0x7ffee26f2aa8(99,99) [line 4]
assign fields 0x7ffee26f2ac8 = 0x7ffee26f2aa8(99,99) [line 5]
HERE 99 99 99 2 [line 7]
destruct 0x7ffee26f2aa8(99,2) [line 8]
destruct 0x7ffee26f2ac8(99,99) [line 8]
```